

6. Project Testing

Project Name

ComicCraft - AI Comic Story Creator using Gemini Models

1. Introduction

Project Testing is performed to ensure that the ComicCraft application works correctly and meets the defined requirements.

Testing helps identify errors, verify individual features, and ensure that the complete application works as expected.

2. Objectives of Testing

- Verify that all project features work correctly.
- Check whether user inputs are processed properly.
- Verify Gemini AI integration.
- Check story generation.
- Verify comic preview.
- Identify and fix errors.
- Ensure a smooth user experience.

3. Testing Methods

3.1 Functional Testing

Functional testing verifies whether each feature performs its intended function.

The following features are tested:

- Story idea input
- Character details
- Story generation
- Gemini AI response
- Comic preview
- Page navigation

3.2 UI Testing

UI testing checks whether the application interface is:

- Easy to understand
- User-friendly
- Properly displayed
- Easy to navigate
- Responsive to user actions

3.3 Integration Testing

Integration testing verifies the communication between:

Frontend ↓ Flask Backend ↓ Gemini AI ↓ Generated Response ↓ Comic Preview

3.4 Error Testing

The application is tested with invalid or incomplete inputs to ensure that appropriate error messages are displayed.

4. Test Cases

Test Case| Input / Action| Expected Result| Status
 TC01| Open application| Home page should load| Pass
 TC02| Enter story idea| Input should be accepted| Pass
 TC03| Enter character details| Details should be accepted| Pass
 TC04| Click Generate| AI story should be generated| Pass
 TC05| Send request to Gemini| Gemini should return a response| Pass
 TC06| View generated story| Story should be displayed| Pass
 TC07| Open comic preview| Comic preview should load| Pass
 TC08| Submit empty input| Appropriate error message should appear| Pass
 TC09| Navigate between pages| Pages should open correctly| Pass
 TC10| Complete workflow| Application should work without major errors| Pass

5. Test Scenarios

Scenario 1: Story Generation

Steps:

1. Open ComicCraft.
2. Enter a story idea.
3. Enter character details.
4. Click the Generate button.
5. Wait for the AI response.

Expected Result: A relevant comic story should be generated.

Scenario 2: Comic Preview

Steps:

1. Generate a story.
2. Open the preview page.
3. View the generated content.

Expected Result: The generated characters, scenes, and dialogues should be displayed correctly.

Scenario 3: Invalid Input

Steps:

1. Leave the required input empty.
2. Click the Generate button.

Expected Result: The application should display an appropriate validation message.

6. Bug Identification and Fixing

During testing, errors may be identified in:

- Frontend pages
- Flask routes
- Gemini API communication
- User input handling
- Story generation
- Comic preview

Identified errors should be corrected and the affected feature should be tested again.

7. Testing Checklist

- Home page works correctly
- Story input works correctly
- Character input works correctly
- Generate button works correctly
- Gemini AI connection works correctly
- Story generation works correctly
- Comic preview works correctly

- Navigation works correctly
- Error handling works correctly
- Complete application workflow works correctly

8. Final Testing Result

The ComicCraft application is tested module by module and as a complete system. The testing process ensures that the major features work correctly and that identified issues are resolved before final submission.

9. Conclusion

Testing is an important phase of the ComicCraft project. It helps ensure that the application is reliable, user-friendly, and meets the defined project requirements.